C++ FÖRDJUPNING

FÖRELÄSNING 3 – FLÖDESSCHEMAN, FLÖDESKONTROLL & DEBUGGNING

AGENDA

- Flödesscheman

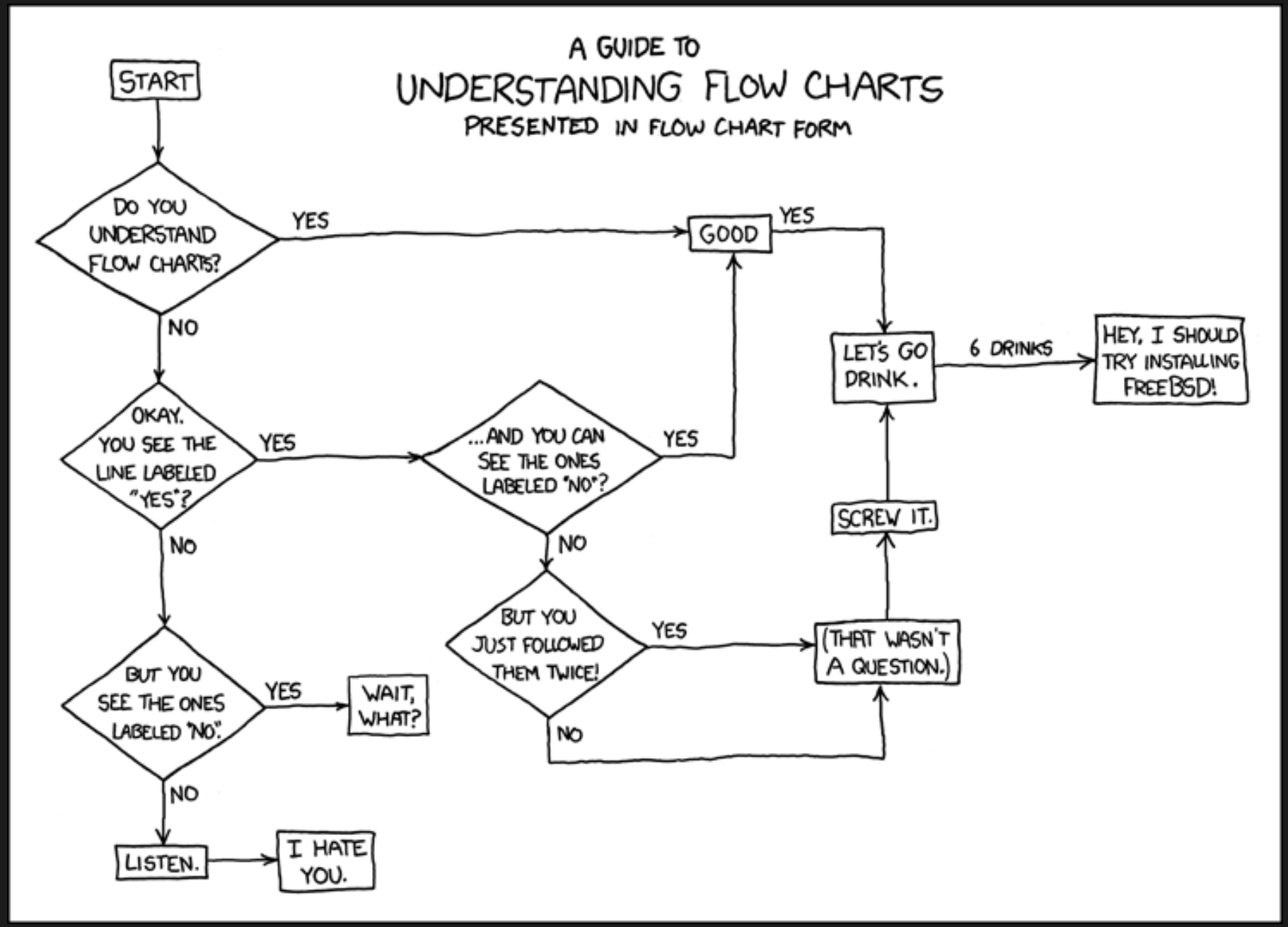
- PAUS

- Flödeskontroll i C++
- Debuggning i C++

C++ FÖRDJUPNING

FLÖDESSCHEMA - EN VÄRLD AV FLÖDEN

TGA


A GUIDE TO
UNDERSTANDING FLOW CHARTS
PRESENTED IN FLOW CHART FORM
START
DO YOU UNDERSTAND FLOW CHARTS?
YES
GOOD
YES
NO
LET'S GO DRINK.
6 DRINKS
HEY, I SHOULD TRY INSTALLING FREEBSD!
OKAY. YOU SEE THE LINE LABELED "YES"?
YES
...AND YOU CAN SEE THE ONES LABELED "NO"?
YES
NO
SCREW IT.
NO
BUT YOU JUST FOLLOWED THEM TWICE!
YES
(THAT WASN'T A QUESTION.)
BUT YOU SEE THE ONES LABELED "NO".
YES
WAIT, WHAT?
NO
NO
LISTEN.
I HATE YOU.

FLÖDESSCHEMA

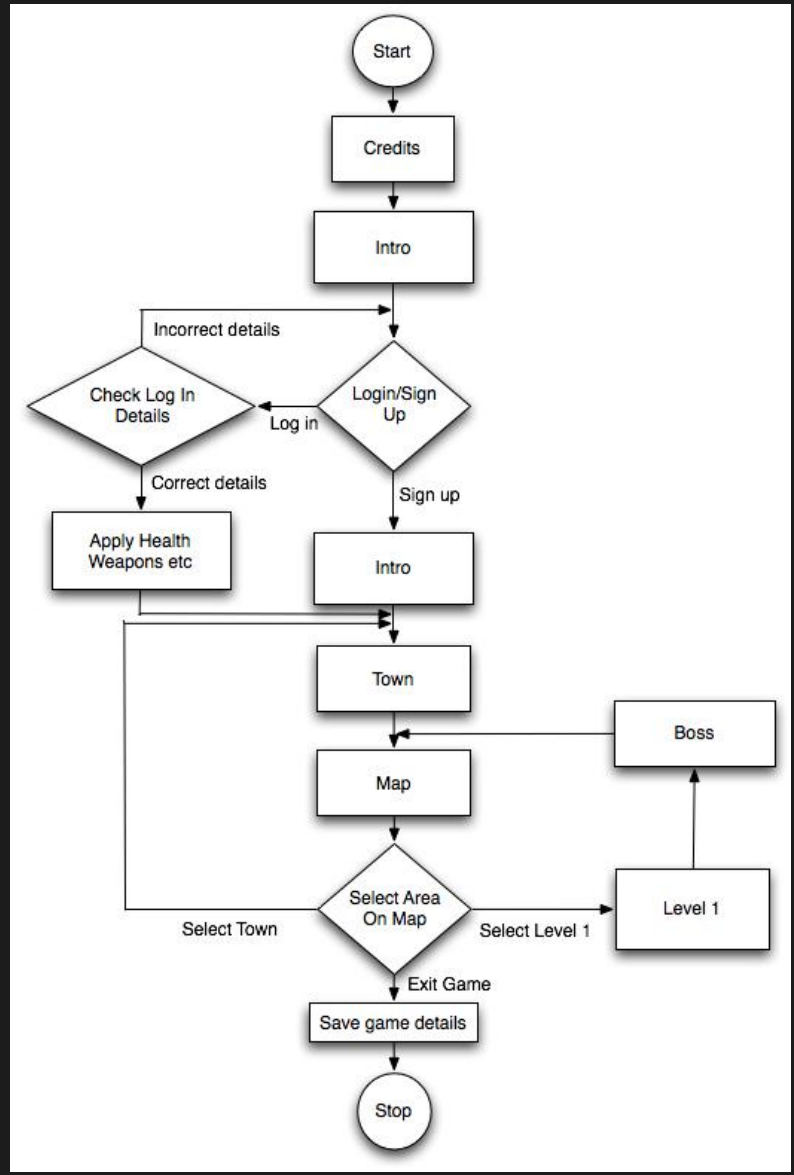
- Ett verktyg för spelutveckling

  - Vid speldesign
    - Mappa spelets flöde
    - Vad gör en spelare i spelet?
    - Hur kan man ta sig igenom en bana?

  - Kodning
    - Planering
    - Bugjakt
    - Kommunicera till andra discipliner eller programmerare

FLÖDESSCHEMA - SPELDESIGN

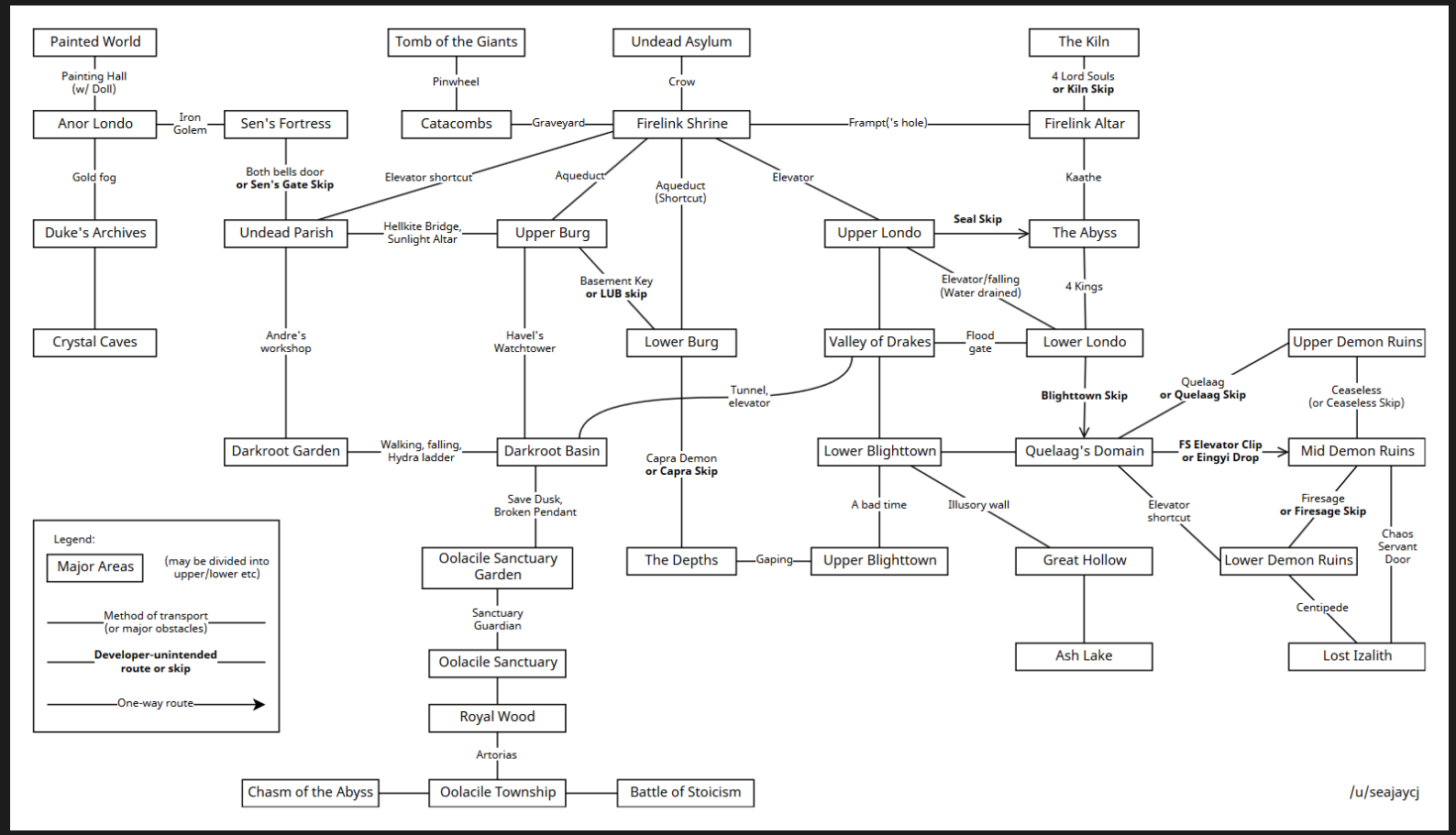
- Flödesschema över olika nivåer
  - Övergripande över spelet
    - States



FLÖDESSCHEMA - SPELDESIGN

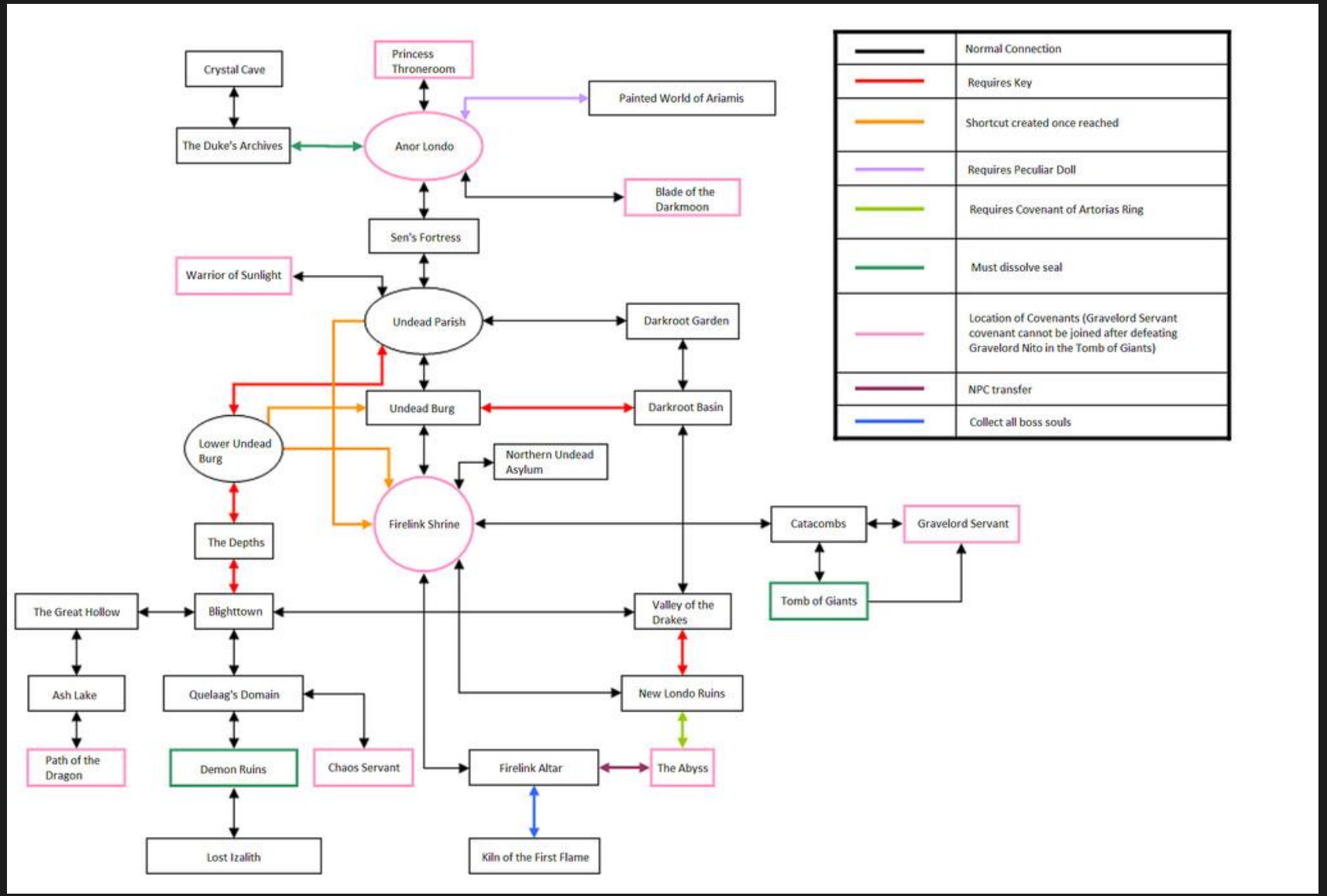
- Komplexa strukturer blir nästan omöjliga att beskriva utan flödesdiagram.

- Open World:
  Hur kan spelaren navigera mellan olika delar i Dark Souls?



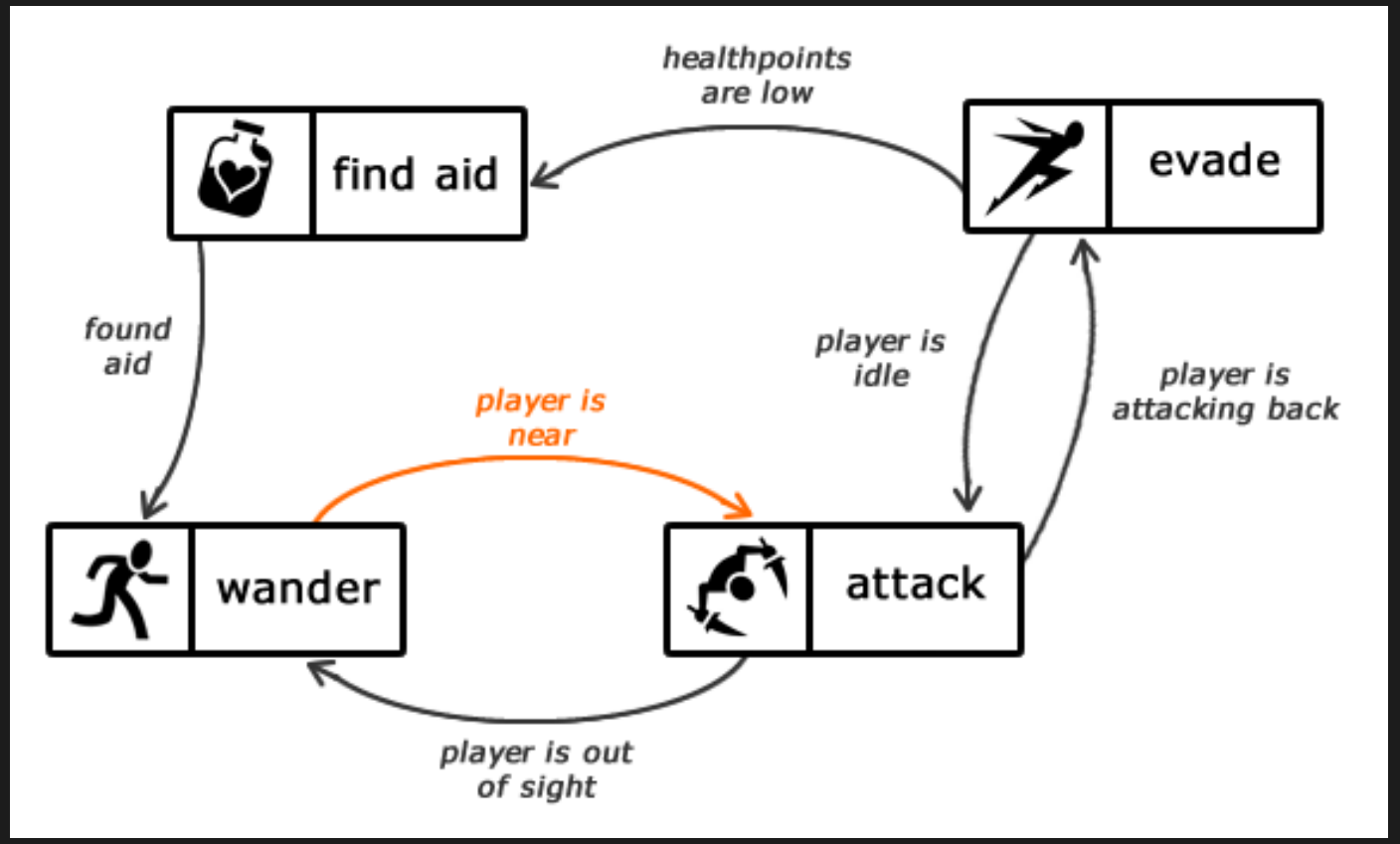
FLÖDESSCHEMA - SPELDESIGN

- Går att göra ännu tydligare.



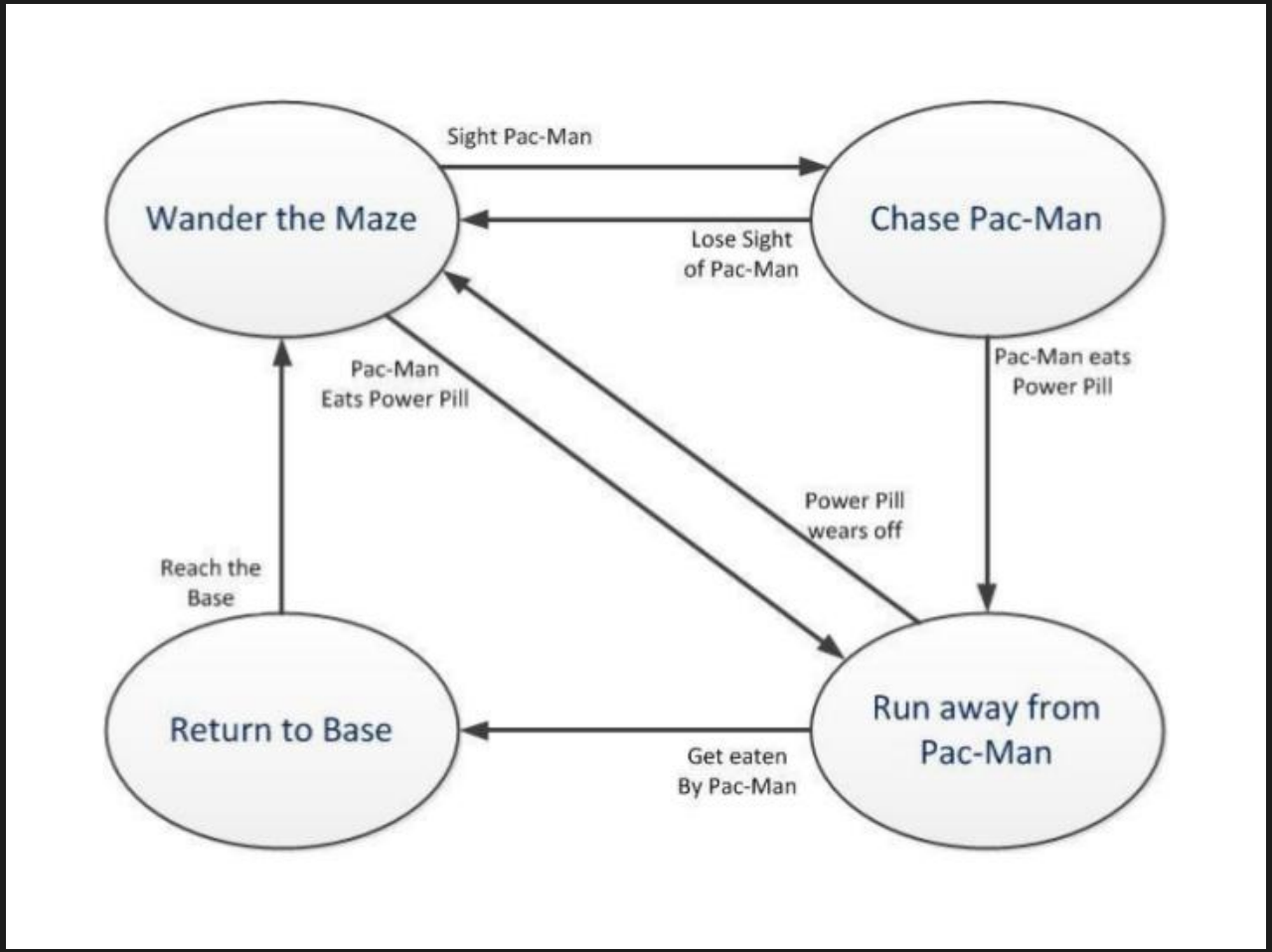
FLÖDESSCHEMA – AI BEHAVIOURS

- Språk som både programmerare och designer förstår.
- Går att översätta direkt till kod på ett enkelt sätt
- Enklare att upptäcka ologiska buggar.



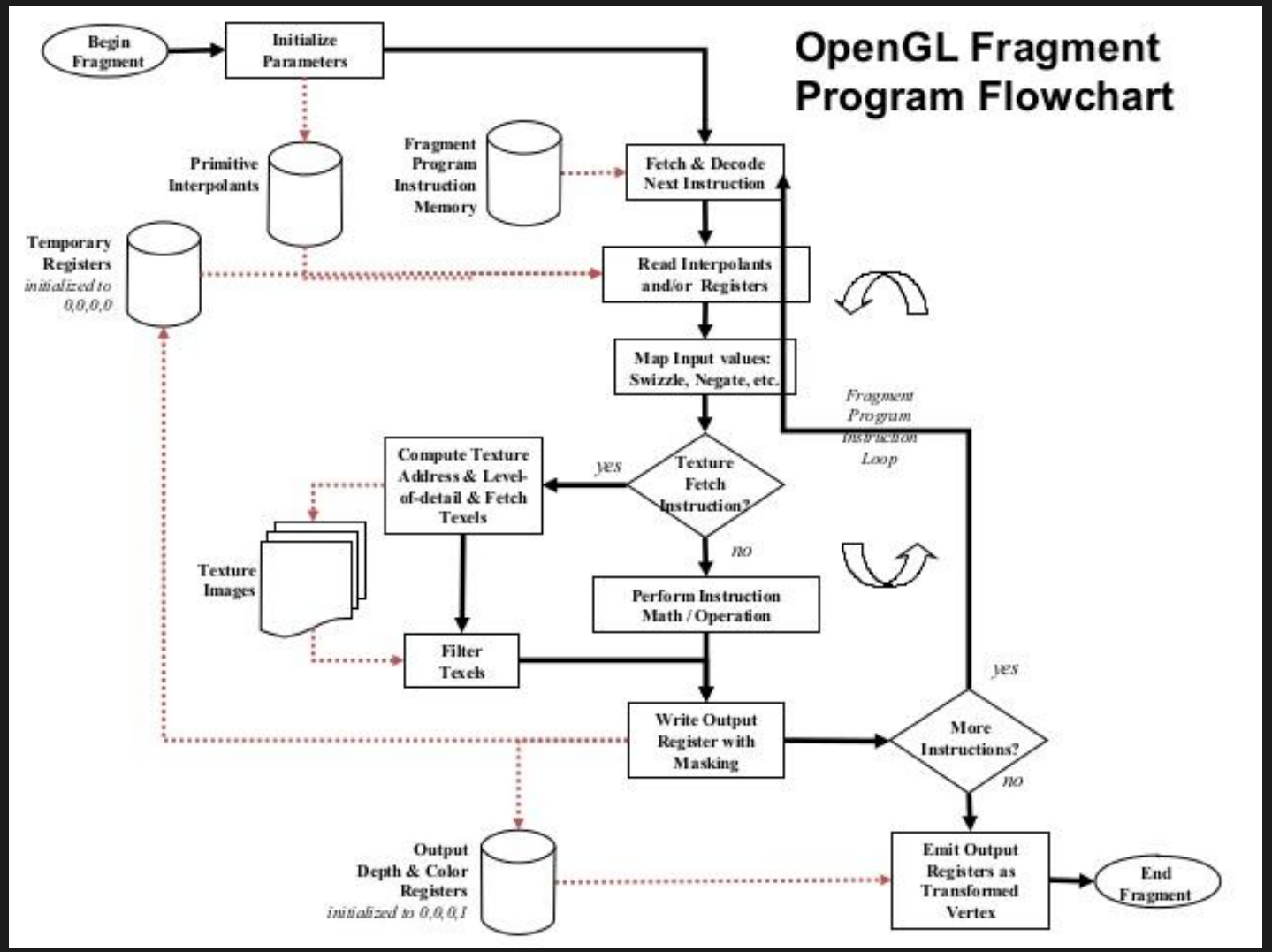
FLÖDESSCHEMA – AI BEHAVIOURS

- Ni kommer arbeta med konceptet i AI kursen i tvåan.



FLÖDESSCHEMA – KOD

- Komplexa algoritmer blir enklare att följa med tydligare flöde.
- Exempel:
  Vad gör openGL med varje pixel som ska ritas ut på skärmen?

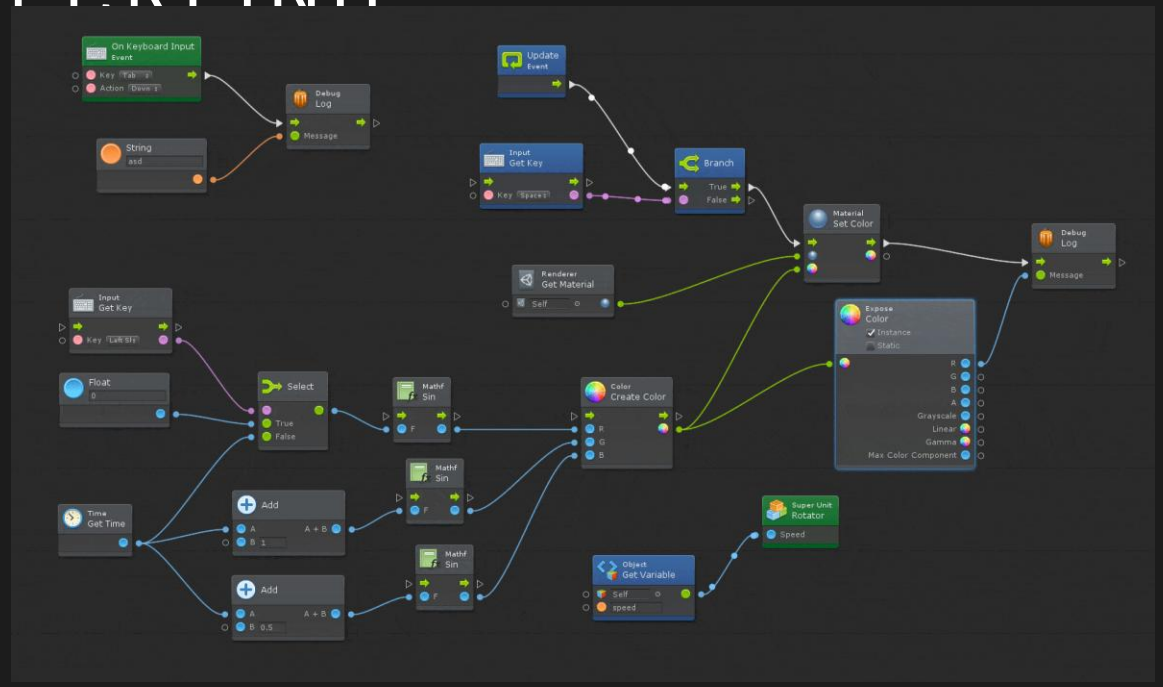


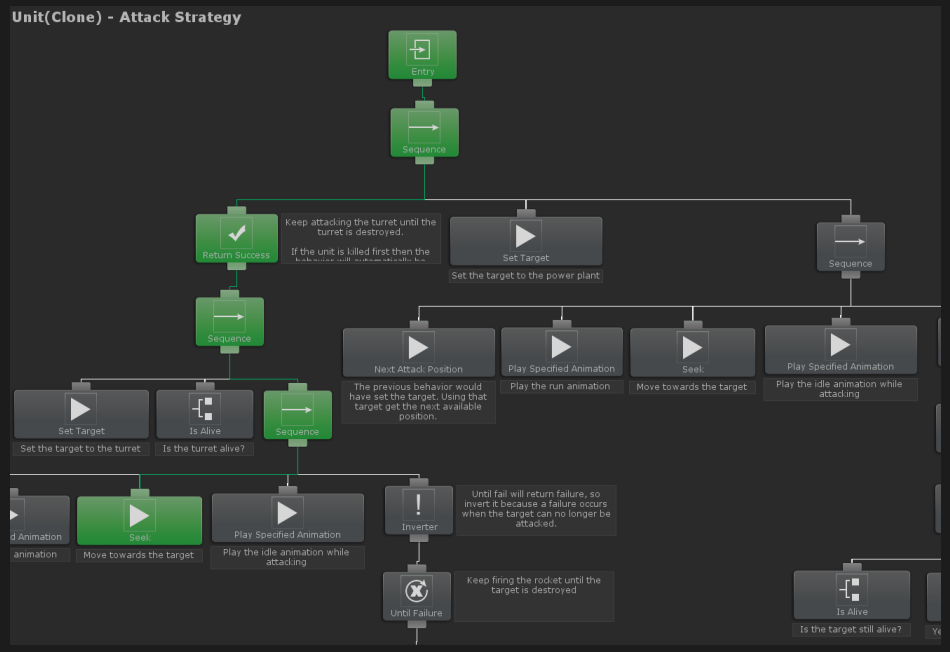
FLÖDESSCHEMA

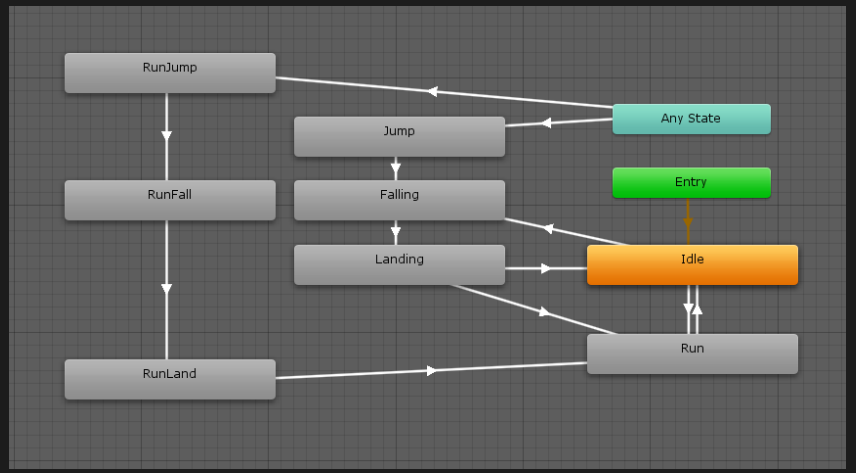
- Finns standard för alla typer av diagram
  - UML
    - Används för klassdiagram
- Används sällan eller aldrig av spelbranschen
  - Bestäm själva vad ni menar med pilar, lådor, cirklar
  - Det viktiga är att alla förstår diagrammet!

FLÖDESSCHEMA - SPELUTVECKLING

- Finns också verktyg som använder flödesscheman direkt
  - Visual-scriptning
  - Behaviour-trees (AI)
  - Animationsträd





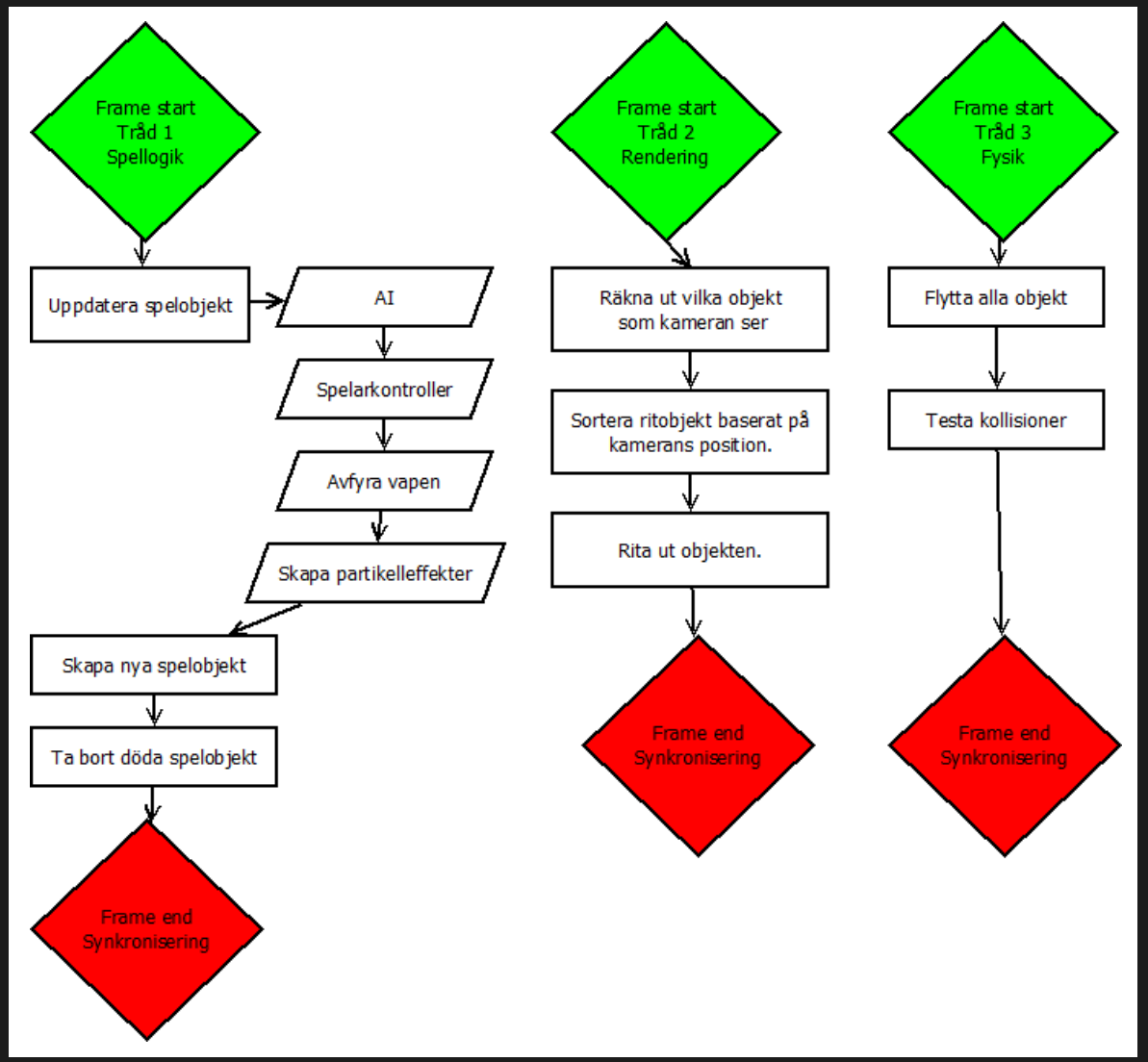


KODENS BEGRÄNSNINGAR

- Får skriva väldigt mycket kod för att kolla om det funkar
- Små felskrivningar sitter hela tiden i vägen för att kolla det övergripande programflödet
- Svårt att gå igenom och dubbelkolla
- Svårt att presentera för andra
- Rörigt med en massa småsaker att ta hänsyn till hela tiden
- Ett enklare verktyg behövs

KODENS BEGRÄNSNINGAR

- Svårt att hålla reda på ett flöde?
- Tänk er tre olika flöde då?
- Som kör i olika hastigheter?
- Svårt att få översikt på kod på flera olika ställen samtidigt.
  - Även om det finns några bra verktyg för det i Visual Studio.
- Lättare att upptäcka problem när man kan se det på högnivå och systemen parallellt.



ED DRAW MINDMAP

- Tre gratisalternativ som är bra:
  - http://www.edrawsoft.com/freemind.php
  - http://dia-installer.de/
  - http://www.draw.io
- Papper funkar så klart lika bra eller ofta bättre
- Whiteboards likaså
- Men praktiskt med digitalt om man behöver inkludera det i ett dokument
- Installera på er datorn om ni vill använda något av dom

KOMMUNICERA DIN IDÉ

- Kommunikation är grunden för allt grupparbete
- Om du inte kan förmedla till andra vad du tycker och tänker så sitter kunskapen fast hos dig
- Teamwork är grunden i spelutveckling.
  - Dokumentation i form av flödesdiagram är användbara då andra ska lära sig systemet

PROBLEMLÖSNING PÅ EN HÖGRE NIVÅ

- Vi vill lösa de stora problemen innan man kommer ner till de små
- Träning i visualisering och logiskt tänkande
- Programkod är bara ett flöde för vad datorn ska göra vid varje steg

TIPS

- För er som är intresserad av Game Design
  - Mark Brown
    - http://www.youtube.com/user/McBacon1337
  - Han använder en version flowcharts flitigt i sin serie Boss Keys

PAUS

C++ FÖRDJUPNING

STATEMENTS

STATEMENTS

- Ett statement är en avgränsad del av ett program
  - Oftast avgränsad av ett `;`
  - Vanligtvis en rad kod i era program
  - Man *kan* skriva flera statements på en rad:
    `int i; i = 4;`
    - Det betyder inte att man ska!
- Vissa statements har andra statements inbakade i sig
  - Kallas block, eller *compound statement*
  - Inleds med `{` och avslutas med `}`

STATEMENTS

- I vissa statements är det tillåtet att skriva variabeldeklarationer
  - Statements bildar då ett scope
  - Variablerna försvinner då scopet avslutas
- `for` och `while` följt av `{}` räknas som ett statement och blir ett scope
  - Deras statement börjar alltså innan `{`
- All kommunikation med C++ går genom statements

TGA

C++ FÖRDJUPNING

FLÖDESKONTROLL

FLÖDESKONTROLL

- Sedan tidigare har vi sett följande:
  - `if`
  - `switch`
  - `for`
  - Range `for`
  - `while`
  - `do… while`
  - Funktioner

FLÖDESKONTROLL

- Vi har även pratat om oändliga loopar
  - Slarvfel i loop-koden
  - Otydliga *exit conditions*
  - Problem med datatyper
    - ex. `vector::size()` ger en `unsigned` (inte en `signed`)

- Vi har även en del andra metoder för flödeskontroll:
  - `break`
  - `continue`
  - `goto`
  - Undantag (*exceptions)*

BREAK

- `break`
  - Används i `switch`-satser för att hoppa ur ett `case`
  - Hoppar ur det närmsta (innersta) `for`-, `while`-, `do`- eller `switch`-statement det ligger i
  - Kan inte hoppa ur nästlade loopar eller `switch` – endast den innersta!
  - Ger ytterligare ett *exit condition*
  - Syns inte lika tydligt som villkoren i ex. `while` och `for`
    - Kan göra koden mer svårläst

CONTINUE

- `continue`
  - Kan skrivas i loopar: `for`, `while`, `do... while`
  - Avbryter det varv vi är på i loopen och går till nästa varv
  - Kan inte fortsätta nästlade loopar – endast den innersta!
  - Som `break`, fast hoppar till nästa varv istället för att avbryta hela loopen
  - Index-uppräkning (i `for-loopar`) sker, och nästa varv påbörjas som om inget har hänt

GOTO

- En etikett (label) skrivs med namnet på etiketten följt av ett kolon
  - `MyLabel:`
  - Precis som `case`:es i `switch-satser`!
  - Markerar en position i koden

- Med `goto` hoppar vi till en etikett
  - Blir lätt förvirrande
  - Allmänt accepterat att man väldigt sällan bör använda det
  - Ni klarar hela utbildningen utan att använda det
  - Kan hoppa ur scopes med goto, kör då destruktorer på saker som går ut scope.

GOTO

- Vanligt argument för `goto` användning
  - Behövs för att hoppa ur/avsluta flera loopar på en gång:

```cpp
#include <iostream>

int main()
{
    int counter = 0;

    for (int i = 0; i < 30; i++)
    {
        for (int j = 0; j < 20; j++)
        {
            for (int h = 0; h < 10; h++)
            {
                std::cout << j * h + 1 << std::endl;
                if (counter > 20)
                {
                    goto nextPartOfProgram;
                }
                counter++;
            }
        }
    }

    nextPartOfProgram:
    //DO STUFF
    std::cout << "DONE!" << std::endl;

    return 0;
}
```

GOTO

- Den bättre lösningen på problemet (goto-fri)

```
void AmazingForLoopFunktion()
{
    int counter = 0;

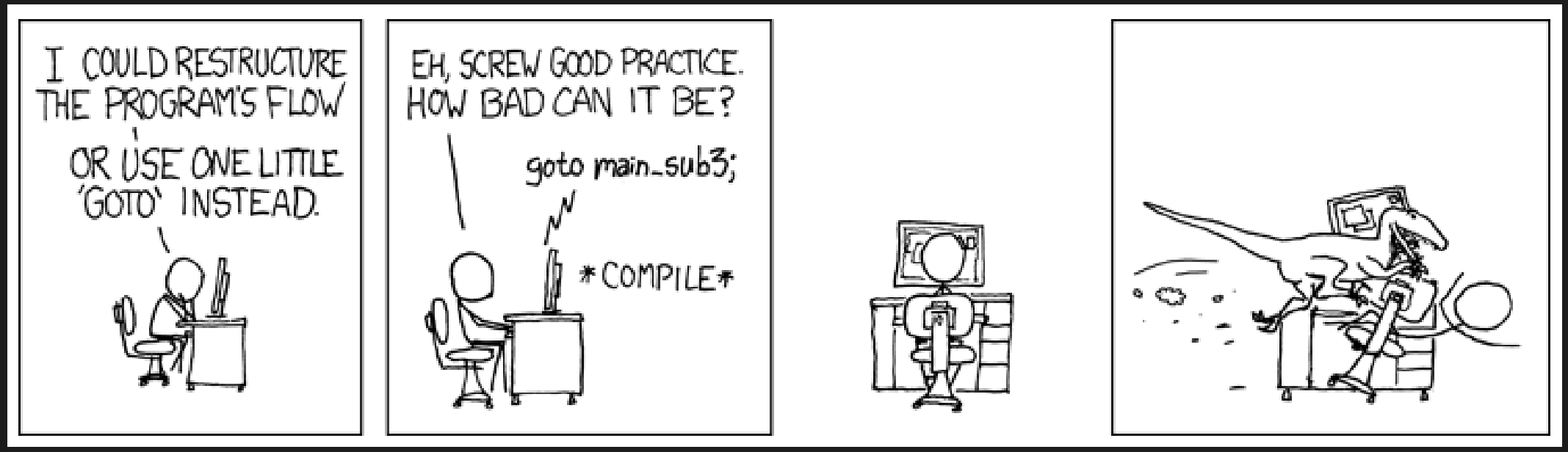
    for (int i = 0; i < 30; i++)
    {
        for (int j = 0; j < 20; j++)
        {
            for (int h = 0; h < 10; h++)
            {
                std::cout << j * h + 1 << std::endl;
                if (counter > 20)
                {
                    return;
                }
                counter++;
            }
        }
    }
}

int main()
{
    AmazingForLoopFunktion();
    //DO STUFF
    std::cout << "DONE!" << std::endl;

    return 0;
}
```

  - Lägg koden i en funktion och avsluta funktionen genom att hoppa ur den.
  - Eller spara en variabel som styr om man ska anropa break på nästa nivå

GOTO


I COULD RESTRUCTURE THE PROGRAM'S FLOW
OR USE ONE LITTLE 'GOTO' INSTEAD.
EH, SCREW GOOD PRACTICE. HOW BAD CAN IT BE?
goto main_sub3;
COMPILE

C++ FÖRDJUPNING

UNDANTAGSHANTERING



UNDANTAGSHANTERING

- Ibland får vi undantag, *exceptions*, i vår kod
  - Detta märker vi ofta som kraschar när vi kör programmet
  - Vi kan hantera dessa för att hantera felen

- Undantagshantering, *exception handling*, består av tre delar:
  - `Try`-block
    - Här lägger vi kod vi vet kan orsaka undantag
  - `Throw`-satser
    - Vi väljer själva att orsaka ett undantag av någon anledning
  - `Catch`-block (ett eller flera)
    - Här hanterar vi de olika undantagen

UNDANTAGSHANTERING

- Vad är syftet?
- Om vi skriver kod så vet vi inte vilken del av programmet som är intresserad, om det går fel.
- Normalt är där två sätt att meddela fel på:
  - Funktion som returnerar felkod
    - T.ex. false om funktionen misslyckades
  - Ett inargument (referens) som sätts beroende på om funktionen lyckades eller ej.

UNDANTAGSHANTERING

- Om det är en lång anropskedja så är det jobbigt att skicka returargumentet hela vägen.
- Dessutom så vill vi kanske använda returvärdet till något annat.

UNDANTAGSHANTERING

- Om vi ska skicka en referens så behövs det skapas en variabel vid varje anrop av funktionen som sedan skickas med.
- Fungerar, men ganska osmidigt och dyrt.

UNDANTAGSHANTERING

- Därför finns undantagshantering.
- Är lite som `goto`, fast som hoppar till närmaste `catch`-block som kan ta emot felet.
  - Och funktionerna som passeras på vägen tillbaka går out of scope.

UNDANTAGSHANTERING

- Vi orsakar ett undantag med hjälp av nyckelordet `throw`
  - Vi kan orsaka undantag av vilka typer vi vill
  - C++ standardbibliotek har error-klasser som vi kan använda

```
throw(5);
throw("Error");
```

- För att hantera ett undantag använder vi ett `catch`-block och anger vilken typ av undantag detta `catch`-block hanterar

```
catch (int err)          // Kan hantera throw(5) men inte throw("Error")
{
    //DO STUFF
}
catch (std::string err) // Kan hantera throw("Error") men inte throw(5)
{
    //DO STUFF
}
```

UNDANTAGSHANTERING

- För att ha ett `catch`-block måste vi först ha ett `try`-block
  - `catch`-blocken kan bara fånga upp undantag som inträffar i try-blocket
- Vi kan ha flera `catch`-block till ett `try`-block för att hantera många olika undantag

```
try
{
    // Kod som kan orsaka throw(5) eller throw("Error")
}
catch (int err)          // Hantera throw(5)
{
    //DO STUFF
}
catch (std::string err) // Hantera throw("Error")
{
    //DO STUFF
}
```

UNDANTAGSHANTERING

- Standardbiblioteket i C++ innehåller ett antal feltyper
  - Inkludera `<stdexcept>`
  - Använd `throw` för att orsaka ett undantag; ange vilket undantag som ska orsakas, exempel: `runtime_error`
  - Dessa feltyper tar in en sträng med felmeddelandet
    - Denna kan vi komma åt i `catch`-blocket genom funktionen `what()`

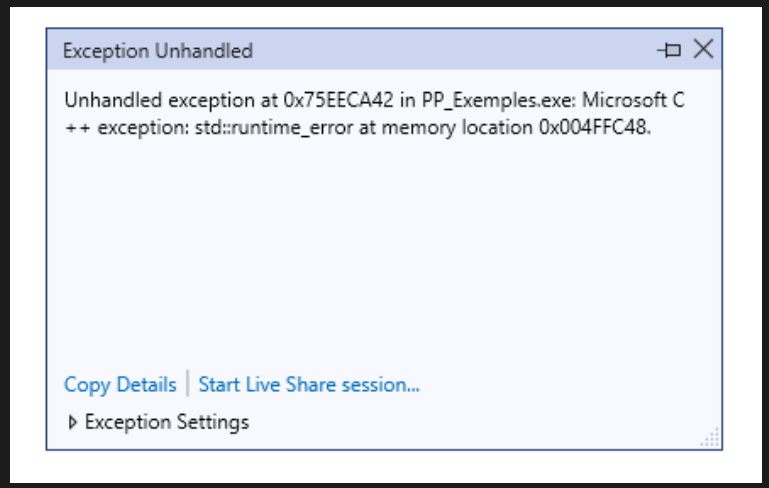
```cpp
#include <stdexcept>

int main()
{
    //SOME CODE

    throw std::runtime_error("ERROR MESSAGE");

    //SOME CODE

    return 0;
}
```



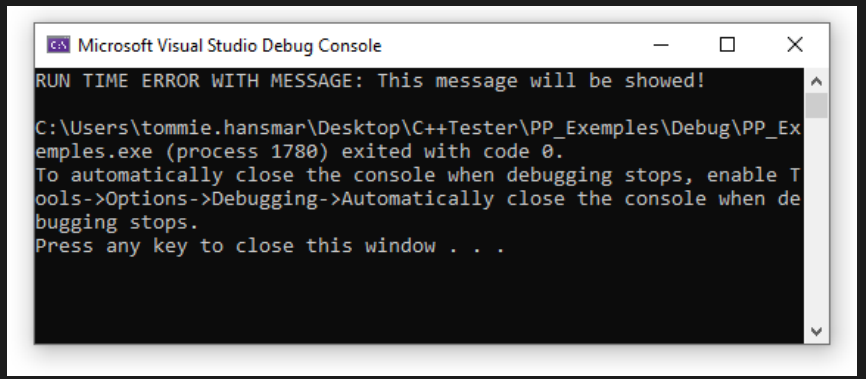
UNDANTAGSHANTERING

- Kod exempel:

```cpp
#include <stdexcept>

int main()
{
    try
    {
        //Kod som kanske orsakar ett undantag
        throw std::runtime_error("This message will be showed!");
    }
    catch (std::runtime_error err)  // Hantera undantag av typen runtime_error
    {
        std::cout << "RUN TIME ERROR WITH MESSAGE: " << err.what() << std::endl;
    }
    catch (std::exception err)      // Hantera alla resterande undantag
    {
        std::cout << "AN ERROR OCCURED WITH MESSAGE: " << err.what() << std::endl;
    }

    return 0;
}
```



UNDANTAGSHANTERING

- Undantagsklasserna:
  - `exception`
  - `runtime_error`
  - `range_error`
  - `overflow_error`
  - `underflow_error`
  - `logic_error`
  - `domain_error`
  - `invalid_argument`
  - `length_error`
  - `out_of_range`
  - `bad_alloc`
  - `bad_cast`

Ligger i header:

`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<stdexcept>`
`<new>`
`<typeinfo>`

UNDANTAGSHANTERING

- Använd inte som ett sätt att skicka information
- Om vi vill fortsätta köra borde det inte varit ett exception
- Om ni har fel bör ni fixa dom, inte ignorera dom.
- Var försiktig med att fånga exceptions och fortsätta köra
- Fånga endast för att bättre rapportera felen

UNDANTAGSHANTERING

- När vi skriver funktioner kan vi markera dem som “noexcept”:
  - `void f() noexcept;`

- Berättar för kompilatorn att den inte kommer kasta exceptions
- Ger kompilatorn möjlighet att optimera bättre

UNDANTAGSHANTERING

- I spelbranschen stängs exceptions ofta av i Retail
  - Då kraschar vi istället bara
- Gör koden lite snabbare
- Används däremot ofta i Debug/Release och för tools
  - Då fångas exceptions, men mest för att ge bra debug-information

C++ FÖRDJUPNING

DEBUGGNING

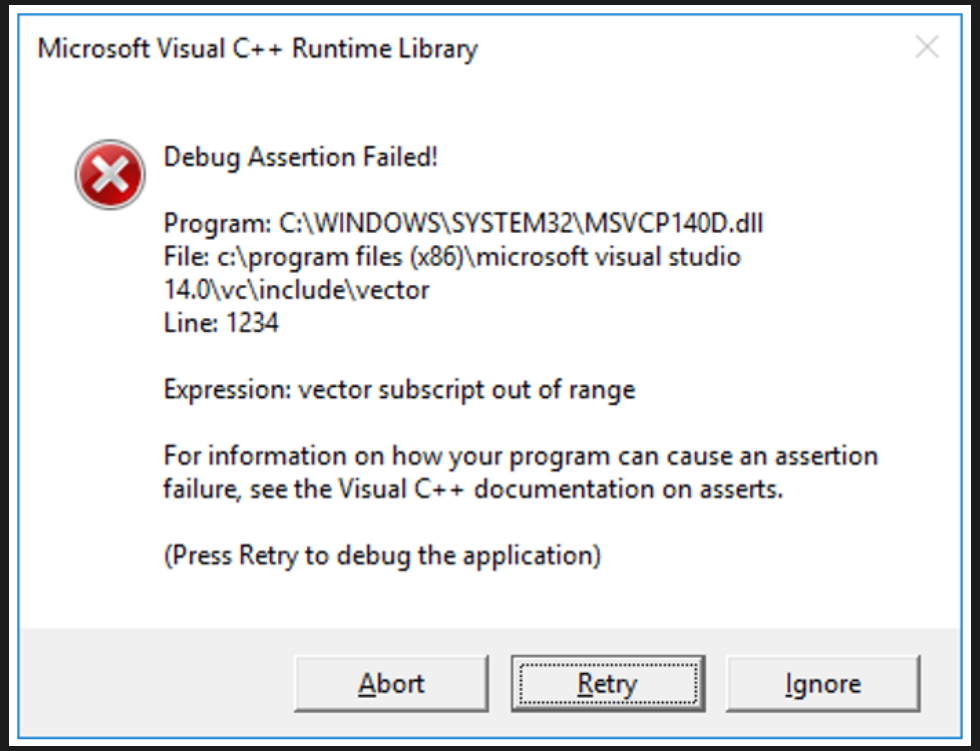
DEBUGGNING

- Repetition – Debugging är jätteviktigt!
  - Ni kommer gå djupare på detta i 2:an
  - Lär er grunderna redan nu

- Steppa igenom er kod
  - F9 - Lägg till/ta bort en breakpoint (på den rad ni är just nu)
  - F10 - Step over (Hoppa till nästa rad)
  - F11 - Step into (Hoppar in i funktion om det finns någon på raden)
  - Shift + F11 - Step out (Kör färdigt funktionen och hoppar ut)

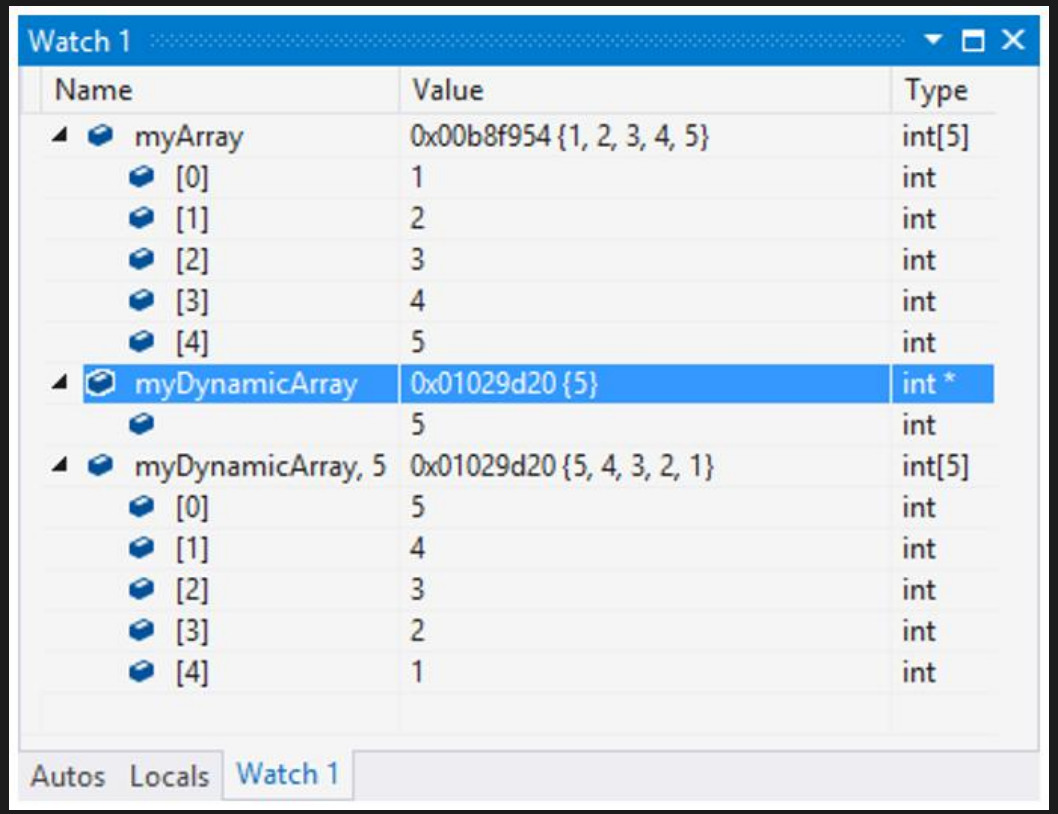
DEBUGGNING

- Om ni får upp detta fönstret när ni kör ert program (oavsett meddelandet)
  - Tryck inte på Abort
  - Tryck på Retry så hoppar koden till felet och du får en chans att se varför det blev fel



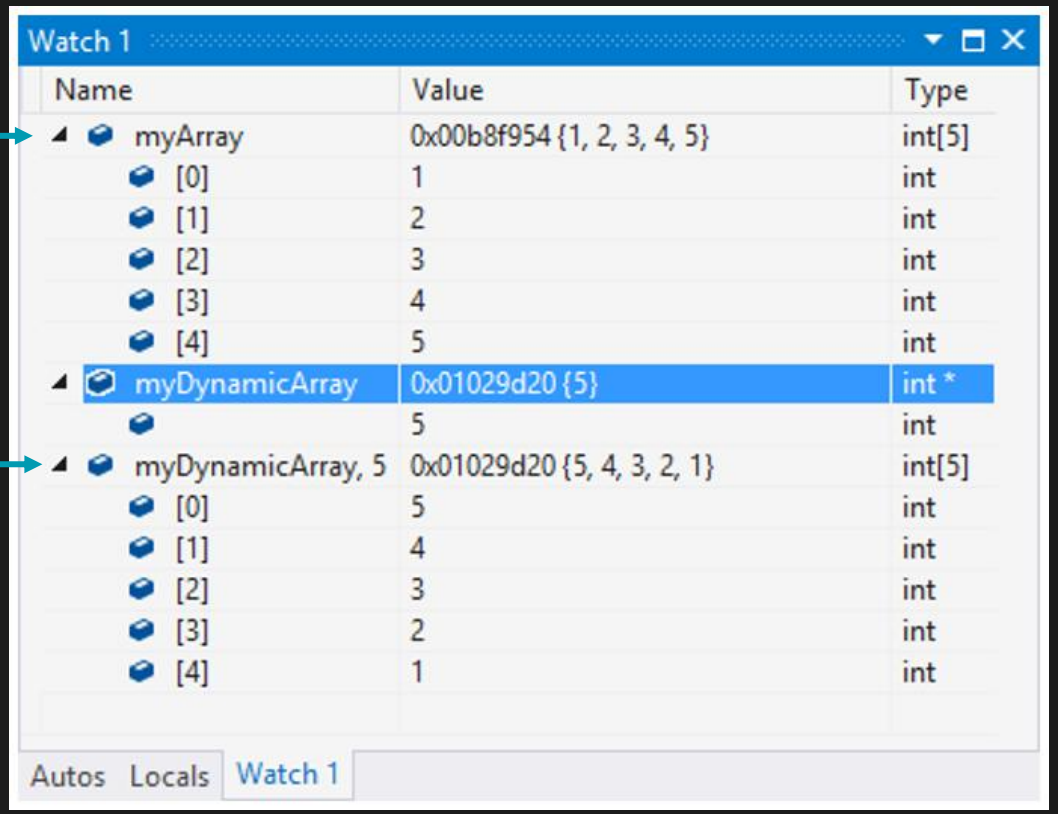
DEBUGGNING

- Watch
  - Ett fönster där du själv kan ange vilka variabler du vill kolla värdet på
    - Högerklicka på en variabel och välj *Add Watch*
    - Skriv in variabelnamnet direkt i fönstret
  - Du kan även ***ändra*** värdet på variablerna
    - Syns bara när vi kör programmet
    - Om programmet är pausat vid en breakpoint



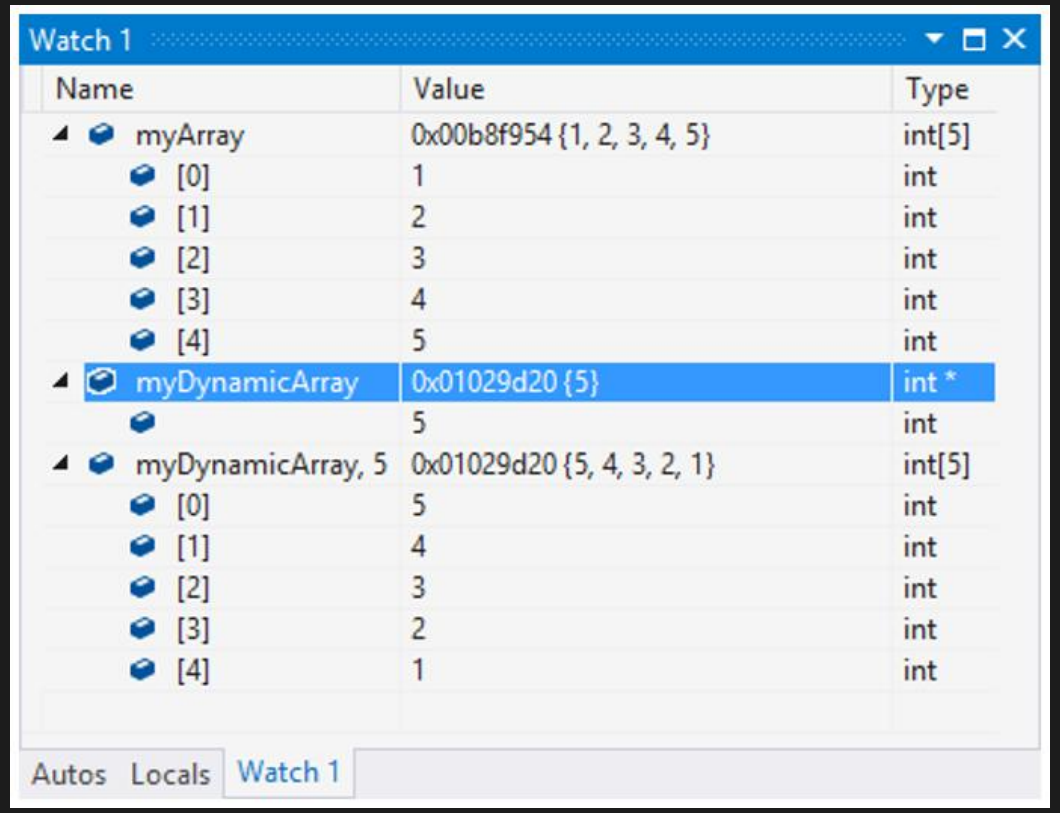
DEBUGGNING

- Watch
  - Vanliga arrayer kan vi lägga till i Watch och se innehållet i utan problem
    - Se *myArray* i bilden
  - För dynamiska arrayer
    - Watch behöver veta hur många element vi vill se för arrayen
      - Skriv ett kommatecken följt av antalet element
    - Se *myDynamicArray* i bilden



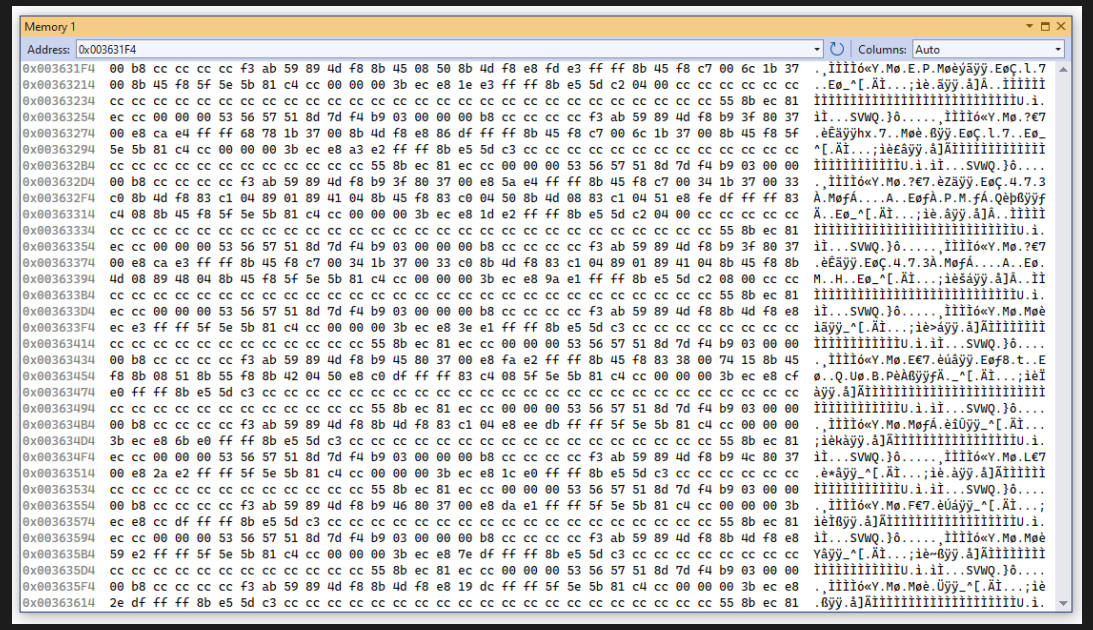
DEBUGGNING

- Watch
  - Det går även att kolla index i ett intervall
    - Skriv +, det första indexet, sedan ett komma och slutligen antalet element
    - Exempel:
      myDynamicArray+1, 2
      skriver ut:
      myDynamicArray[1],
      myDynamicArray[2]



DEBUGGNING

- Vi kan även läsa från en specifik minnesadress
  - Till exempel när vi går till ett scope där variabeln inte finns
  - Vi kan skriva (datatyp*)minnesadress i Watch så visas värdet på adressen
- Vi kan även öppna ett Memoryfönster (under körning) (*Debug -> Windows -> Memory*)
  - I sökfältet skriver vi in minnes-adressen och kan då se vad som ligger i minnet samt om det ändras



DEBUGGNING

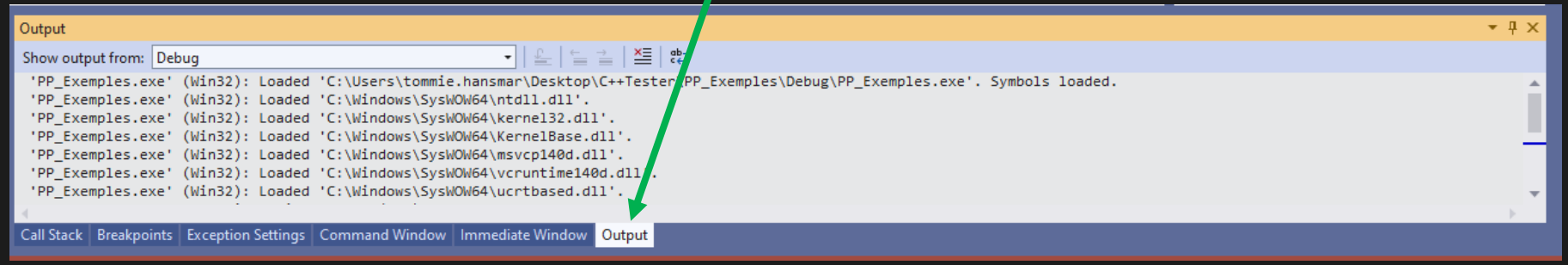
- Skriv om kod
  - Att debugga grötig kod kan vara väldigt svårt
  - Ofta går det snabbare att städa upp ful och grötig kod först och debugga sen
  - Ibland löser sig buggarna av uppstädningen

- Typiska grötighetsfel:
  - Har två variabler som heter nästan samma sak
  - Har en funktion/variabler som heter en sak men inte används till det
  - Återanvänder variabler till olika saker
  - Har samma data sparat på olika ställen som inte alltid matchar

DEBUGGNING

- Stega igenom er kod vid fel!
  - Sätt breakpoints
  - Stega igenom koden
  - Så hittar ni 99% av alla misstag ni gör
  - Ju mer ni lär er använda ovanstående tips, ju snabbare går det att hitta felen

- Output är bra, och underskattad!



DEBUGGNING

- Har ert program hängt sig?
  - Stäng inte av det, tryck på pause så ser ni var programmet befinner sig!



DEBUGGNING

- Lös varningar!
  - Se till att alltid hålla varningar borta
    - Framförallt i spelprojekten!
  - För många varningar leder till att man inte läser dem
    - Då missar man kritiska varningar!

KOMPILERINGSFEL

- Svårt att förstå ett kompileringfel?

- Kolla output-fönstret
- Mer detaljer än error-fönstret!

C++ FÖRDJUPNING

LÄSBARHET

LÄSBARHET

- Det som känns självklart för dig idag kommer att förvirra någon annan (eller dig själv) imorgon
  - Man glömmer **fort** bort koden!
  - Ofta hittar folk inte i den kod de ska modifiera eller underhålla
- Var tydlig
- Följ alltid kodstandard

LÄSBARHET

- Var tydlig!
  - Tydliga namn
  - Använd gärna parenteser i uttryck så att det blir tydligare vad du är ute efter.
    - Exempel:
      (2 + 1) – (3 * 4)
    - Bättre än:
      2 + 1 – 3 * 4 även om resultatet är detsamma

LÄSBARHET

- Använd extra lokala variabler för att ge namn åt mellanresultat
  - Självdokumenterande kod
  - Lättare att debugga
  - I release kommer de ändå optimeras bort så bekymra er inte över prestanda/minne.
- Bryt ut saker i hjälp-funktioner med tydliga namn
- Använd alltid konstanter istället för magiska tal
  - Magiska tal: Hårdkodade tal som inte har en uppenbar betydelse

LÄSBARHET

- Bättre med tydliga enkla läsbara algoritmer
  - Undvik komplicerade bithacks
  - Om du måste göra något komplicerat, använd kommentarer

- ”*Programs must be written for people to read and only incidentally for machines to execute*” – Harold Abelson

C++ FÖRDJUPNING

AVSLUTNINGSVIS

LÄSNING / UPPGIFTEN

- Läs inför nästa föreläsning
  - Ingen läsning

- Uppgift
  - Uppgift 2 – Flappy Helicopter